In [2]:
```
# --- 1. Setup and Configuration ---
import warnings
warnings.filterwarnings("ignore")

import os
import sys
import pandas as pd
import numpy as np
from PIL import Image
from tqdm.auto import tqdm
import joblib

# PyTorch and Deep Learning Libraries
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from transformers import AutoModel, AutoTokenizer
from efficientnet_pytorch import EfficientNet
from sklearn.metrics import classification_report, f1_score

# Set up paths and device
tqdm.pandas()

DATA_DIR = '../data/'
IMAGES_DIR = os.path.join(DATA_DIR, 'images')
RESULTS_DIR = '../results/'
MODELS_DIR = os.path.join(RESULTS_DIR, 'models')
os.makedirs(MODELS_DIR, exist_ok=True)

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Using device: {DEVICE}")

# Feature dimensions
TEXT_DIM = 768    # DistilBERT CLS token
IMAGE_DIM = 1280  # EfficientNet-B0 final pooling
FUSION_DIM = TEXT_DIM + IMAGE_DIM # 2048
HIDDEN_DIM = 512
NUM_CLASSES = 1
BATCH_SIZE = 64
EPOCHS = 10
LEARNING_RATE = 1e-4
```

```
Using device: cpu
```

In [3]:
```
# --- 2. Load DataFrames ---
# Assuming your data files (created in the setup notebook) are available
try:
    df_train = pd.read_csv(os.path.join(DATA_DIR, 'multimodal_train.tsv'), sep='
    df_val = pd.read_csv(os.path.join(DATA_DIR, 'multimodal_validate.tsv'), sep=
    df_test = pd.read_csv(os.path.join(DATA_DIR, 'multimodal_test_public.tsv'),

    print(f"Loaded datasets: Train={len(df_train)}, Val={len(df_val)}, Test={len

    # Filter to only keep rows where image was successfully downloaded
    def filter_downloaded(df):
        return df[df['id'].progress_apply(lambda x: os.path.exists(os.path.join(

    df_train_filtered = filter_downloaded(df_train)
```

```
    df_val_filtered = filter_downloaded(df_val)
    df_test_filtered = filter_downloaded(df_test)

    print(f"Filtered datasets: Train={len(df_train_filtered)}, Val={len(df_val_f

except FileNotFoundError:
    print("ERROR: Data files not found. Please run the setup/download notebooks
    sys.exit()
```

```
Loaded datasets: Train=564000, Val=59342, Test=59319
100%|████████████████████████████████████| 564
000/564000 [00:14<00:00, 38550.13it/s]
100%|█████████████████████████████████████| 5
9342/59342 [00:02<00:00, 24385.05it/s]
100%|█████████████████████████████████████| 5
9319/59319 [00:02<00:00, 25251.34it/s]
Filtered datasets: Train=27046, Val=28358, Test=28330
```

In [4]:
```
# --- 3. Feature Extractor Initialization ---
def initialize_feature_extractors():
    # Text Model (DistilBERT)
    text_model = AutoModel.from_pretrained("distilbert-base-uncased").to(DEVICE)
    text_tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
    text_model.eval()

    # Image Model (EfficientNet-B0)
    image_model = EfficientNet.from_pretrained('efficientnet-b0').to(DEVICE)
    image_model._fc = nn.Identity()
    image_model.eval()

    return text_model, text_tokenizer, image_model

TEXT_MODEL, TEXT_TOKENIZER, IMAGE_MODEL = initialize_feature_extractors()
print("Feature Extractors Initialized.")
```

```
Loaded pretrained weights for efficientnet-b0
Feature Extractors Initialized.
```

In [5]:
```
# --- 4. Feature Extraction Helpers ---

def extract_text_feature(text: str) -> np.ndarray:
    inputs = TEXT_TOKENIZER(text, return_tensors="pt", truncation=True, padding=
    with torch.no_grad():
        outputs = TEXT_MODEL(**inputs)
        # Use CLS token feature
        feature = outputs.last_hidden_state[:, 0, :].cpu().numpy()  # [1, 768]
    return feature

def extract_image_feature(image_path: str) -> np.ndarray:
    try:
        image = Image.open(image_path).convert("RGB").resize((224, 224))
    except:
        # Fallback for corrupted images, return array of NaNs or zeros
        return np.zeros((1, IMAGE_DIM), dtype=np.float32)

    image_tensor = torch.tensor(np.array(image)).permute(2, 0, 1).float() / 255.
    # Simple normalization
    image_tensor = (image_tensor - 0.5) / 0.5
    image_tensor = image_tensor.unsqueeze(0).to(DEVICE)
```

```
    with torch.no_grad():
        feature = IMAGE_MODEL(image_tensor).cpu().numpy()  # [1, 1280]
    return feature

def extract_fused_features(df, split_name):
    # Define file path for caching
    cache_path = os.path.join(RESULTS_DIR, f'{split_name}_fused_features.pkl')

    if os.path.exists(cache_path):
        print(f"Loading cached features for {split_name}...")
        X, y = joblib.load(cache_path)
        return X, y

    print(f"Extracting features for {split_name} ({len(df)} samples)...")

    fused_features = []
    labels = df['2_way_label'].values

    for row in tqdm(df.itertuples(), total=len(df)):
        text_feat = extract_text_feature(row.clean_title).flatten()
        image_path = os.path.join(IMAGES_DIR, f'{row.id}.jpg')
        image_feat = extract_image_feature(image_path).flatten()

        # Concatenate features: Early Fusion
        fused_feat = np.concatenate([text_feat, image_feat])
        fused_features.append(fused_feat)

    X = np.array(fused_features)
    y = labels

    print(f"Features extracted. Shape X: {X.shape}, Shape y: {y.shape}")
    joblib.dump((X, y), cache_path)
    print(f"Features cached to {cache_path}")
    return X, y

# Extract and cache features for all splits
X_train, y_train = extract_fused_features(df_train_filtered, 'train')
X_val, y_val = extract_fused_features(df_val_filtered, 'val')
X_test, y_test = extract_fused_features(df_test_filtered, 'test')
```

```
Extracting features for train (27046 samples)...
100%|
| 27046/27046 [58:03<00:00,  7.76it/s]
Features extracted. Shape X: (27046, 2048), Shape y: (27046,)
Features cached to ../results/train_fused_features.pkl
Extracting features for val (28358 samples)...
100%|
28358/28358 [1:06:42<00:00,  7.08it/s]
Features extracted. Shape X: (28358, 2048), Shape y: (28358,)
Features cached to ../results/val_fused_features.pkl
Extracting features for test (28330 samples)...
100%|
28330/28330 [7:47:29<00:00,  1.01it/s]
Features extracted. Shape X: (28330, 2048), Shape y: (28330,)
Features cached to ../results/test_fused_features.pkl
```

In [6]:

```
# --- 5. Deep Fusion Network (DFN) Definition ---

class DeepFusionNetwork(nn.Module):
```

```
    """
    Multi-Layer Perceptron (MLP) for classifying fused features.
    """
    def __init__(self, input_dim=FUSION_DIM, hidden_dim=HIDDEN_DIM, output_dim=N
        super(DeepFusionNetwork, self).__init__()

        # 1. Initial Projection Layer (Input: 2048 -> Hidden: 512)
        self.fc1 = nn.Linear(input_dim, hidden_dim)

        # 2. Regularization
        self.dropout = nn.Dropout(p=dropout_rate)

        # 3. Final Classification Layer (Hidden: 512 -> Output: 1)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, features):
        # features shape: (B, 2048)
        x = F.relu(self.fc1(features))
        x = self.dropout(x)
        # Output is raw logits, Binary Cross Entropy with Logits will handle the
        logits = self.fc2(x)
        return logits

# Convert NumPy arrays to PyTorch tensors
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)

X_val_t = torch.tensor(X_val, dtype=torch.float32)
y_val_t = torch.tensor(y_val, dtype=torch.float32).unsqueeze(1)

# Create DataLoaders
train_dataset = TensorDataset(X_train_t, y_train_t)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)

val_dataset = TensorDataset(X_val_t, y_val_t)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

In [7]:

```
# --- 6. PyTorch Training Loop ---

model = DeepFusionNetwork().to(DEVICE)
criterion = nn.BCEWithLogitsLoss() # Combines Sigmoid and Binary Cross Entropy
optimizer = torch.optim.Adam(model.parameters(), lr=LEARNING_RATE)

def evaluate(model, data_loader, criterion):
    model.eval()
    total_loss = 0
    all_preds = []
    all_labels = []
    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            total_loss += loss.item() * X_batch.size(0)

            # Convert logits to probabilities and then to predictions
            probs = torch.sigmoid(logits)
            preds = (probs > 0.5).int().cpu().numpy()
            all_preds.extend(preds)
            all_labels.extend(y_batch.int().cpu().numpy())
```

```python
    avg_loss = total_loss / len(data_loader.dataset)
    f1_macro = f1_score(all_labels, all_preds, average='macro')
    return avg_loss, f1_macro

best_val_f1 = -1
best_model_path = os.path.join(MODELS_DIR, 'deep_fusion_model.pth')

print("Starting DFN Training...")
for epoch in range(EPOCHS):
    model.train()
    train_loss = 0
    for X_batch, y_batch in tqdm(train_loader, desc=f"Epoch {epoch+1}/{EPOCHS} (
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)

        optimizer.zero_grad()
        logits = model(X_batch)
        loss = criterion(logits, y_batch)
        loss.backward()
        optimizer.step()

        train_loss += loss.item() * X_batch.size(0)

    avg_train_loss = train_loss / len(train_loader.dataset)

    # Validation
    val_loss, val_f1 = evaluate(model, val_loader, criterion)

    print(f"Epoch {epoch+1} | Train Loss: {avg_train_loss:.4f} | Val Loss: {val_

    # Save the best model based on Validation F1 Score
    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        torch.save(model.state_dict(), best_model_path)
        print(f"--> Saved best model with Val F1: {best_val_f1:.4f} to {best_mod

print("Training Complete.")
```

```
Starting DFN Training...
Epoch 1/10 (Train): 100%|████████████████████████████
████| 423/423 [00:05<00:00, 72.04it/s]
Epoch 1 | Train Loss: 0.4698 | Val Loss: 0.4062 | Val Macro F1: 0.7970
--> Saved best model with Val F1: 0.7970 to ../results/models\deep_fusion_model.p
th
Epoch 2/10 (Train): 100%|████████████████████████████
████| 423/423 [00:04<00:00, 85.60it/s]
Epoch 2 | Train Loss: 0.3905 | Val Loss: 0.3860 | Val Macro F1: 0.8155
--> Saved best model with Val F1: 0.8155 to ../results/models\deep_fusion_model.p
th
Epoch 3/10 (Train): 100%|████████████████████████████
████| 423/423 [00:04<00:00, 88.97it/s]
Epoch 3 | Train Loss: 0.3668 | Val Loss: 0.3739 | Val Macro F1: 0.8208
--> Saved best model with Val F1: 0.8208 to ../results/models\deep_fusion_model.p
th
Epoch 4/10 (Train): 100%|████████████████████████████
████| 423/423 [00:04<00:00, 88.71it/s]
Epoch 4 | Train Loss: 0.3488 | Val Loss: 0.3659 | Val Macro F1: 0.8275
--> Saved best model with Val F1: 0.8275 to ../results/models\deep_fusion_model.p
th
```

```
Epoch 5/10 (Train): 100%|
| 423/423 [00:04<00:00, 89.43it/s]
Epoch 5 | Train Loss: 0.3330 | Val Loss: 0.3603 | Val Macro F1: 0.8308
--> Saved best model with Val F1: 0.8308 to ../results/models\deep_fusion_model.p
th
Epoch 6/10 (Train): 100%|
| 423/423 [00:04<00:00, 88.73it/s]
Epoch 6 | Train Loss: 0.3182 | Val Loss: 0.3553 | Val Macro F1: 0.8316
--> Saved best model with Val F1: 0.8316 to ../results/models\deep_fusion_model.p
th
Epoch 7/10 (Train): 100%|
| 423/423 [00:04<00:00, 92.14it/s]
Epoch 7 | Train Loss: 0.3049 | Val Loss: 0.3525 | Val Macro F1: 0.8332
--> Saved best model with Val F1: 0.8332 to ../results/models\deep_fusion_model.p
th
Epoch 8/10 (Train): 100%|
| 423/423 [00:06<00:00, 70.06it/s]
Epoch 8 | Train Loss: 0.2909 | Val Loss: 0.3490 | Val Macro F1: 0.8368
--> Saved best model with Val F1: 0.8368 to ../results/models\deep_fusion_model.p
th
Epoch 9/10 (Train): 100%|
| 423/423 [00:04<00:00, 89.03it/s]
Epoch 9 | Train Loss: 0.2780 | Val Loss: 0.3457 | Val Macro F1: 0.8388
--> Saved best model with Val F1: 0.8388 to ../results/models\deep_fusion_model.p
th
Epoch 10/10 (Train): 100%|
| 423/423 [00:04<00:00, 93.13it/s]
Epoch 10 | Train Loss: 0.2664 | Val Loss: 0.3487 | Val Macro F1: 0.8367
Training Complete.
```

In [8]:

```python
# --- 7. Final Test Evaluation and Report ---

print("\n--- Evaluating Best DFN Model on Test Set ---")

# Load the best model
model.load_state_dict(torch.load(best_model_path, map_location=DEVICE))
model.eval()

# Prepare Test DataLoader
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
test_dataset = TensorDataset(X_test_t, y_test_t)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

# Run evaluation
test_loss, test_f1 = evaluate(model, test_loader, criterion)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Macro F1-Score: {test_f1:.4f}")

# Detailed Classification Report
all_preds = []
all_labels = []
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
        logits = model(X_batch)
        probs = torch.sigmoid(logits)
```

```python
        preds = (probs > 0.5).int().cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(y_batch.int().cpu().numpy())

all_preds = np.array(all_preds).flatten()
all_labels = np.array(all_labels).flatten()

report = classification_report(all_labels, all_preds, target_names=['FAKE', 'REA
print("\nClassification Report:\n")
print(report)

# Save the final report
report_path = os.path.join(RESULTS_DIR, 'deep_fusion_report.txt')
with open(report_path, 'w') as f:
    f.write(report)
print(f"\nFinal classification report saved to {report_path}")
```

```
--- Evaluating Best DFN Model on Test Set ---
Test Loss: 0.3461
Test Macro F1-Score: 0.8379

Classification Report:

              precision    recall  f1-score   support

        FAKE     0.8703    0.8798    0.8750     17314
        REAL     0.8078    0.7939    0.8008     11016

    accuracy                         0.8464     28330
   macro avg     0.8391    0.8369    0.8379     28330
weighted avg     0.8460    0.8464    0.8462     28330


Final classification report saved to ../results/deep_fusion_report.txt
```